Security & Privacy — Questions from CISOs and DPOs

For security and privacy leadership: agent authority, consent, data custody, and audit — and what you can **verify yourself** without trusting us.

`Last updated: 2026-07-14 · Worker version: d2065f3d · This document is versioned and public.`

Before you start. You are probably assuming that a system built on Webflow, Xano, and Cloudflare is less secure than one built on Salesforce. That is a reasonable prior. This document gives you the specifics so you can decide for yourself rather than on the shape of the logos. We publish our open findings, with dates, at the bottom. **If you only read one section, read that one.**

---

1 · THE CONSENT RECORD

Who can write to the consent ledger? Only an authenticated subject, writing their own record. Every write requires a valid login token; the subject id is bound to that token server-side, so a forged or client-supplied id cannot record consent for anyone else (a mismatch is rejected). Consent given *before* login — at the cookie banner, before a subject has a session — is applied immediately client-side via Consent Mode v2 and written to the ledger only when the subject authenticates. The pre-session banner never makes an unauthenticated server write.

Verify it yourself: an unauthenticated `POST https://crm-sync.dev/auth/consent-sync` returns **401**. The thing you can run beats the thing you have to believe.

Can a record be modified or deleted after the fact? The ledger is append-only. There is no update or delete path on a written record — corrections are new records, not edits.

Can you prove a record wasn't altered? Not yet, cryptographically. Records are append-only by application design, not by hash chain. Chaining lands in Sprint 2 — see §5. Until then, we do not use the words "tamper-evident" for the consent log.

What does a record contain? Subject, consent type, action (granted/revoked), method (banner, signup, compliance page), the policy version in force at the time, the user agent, the GA4 session id, the client timestamp, and the server timestamp. That is more provenance than the major marketing platforms capture.

Can we reconstruct the consent posture as of a past date? Yes. The record is a sequence of state transitions, not a current-value field.

Can we export it and read it without you? Yes. It is your data in your Xano instance, exportable in standard formats. You do not need us to read your own audit trail.

What happens to the record if we stop paying? It is still yours. Your compliance evidence is not a subscription. If an audit trail can be lost by missing an invoice, it was never an audit trail. (This is not true of every platform in this category — it is worth asking your other vendors the same question.)

---

2 · ENFORCEMENT

Where is consent actually enforced? Server-side, at the point of transmission. Every outbound push to every connected platform checks the subject's consent state before sending.

What happens if the consent state is unknown? We don't send. The system defaults to denied. Records that lack a consent basis are stored but not projected outward.

This is the distinction we'd ask you to hold onto: a system that *records* consent is not the same as a system that *enforces* it. Most platforms model consent well and have no runtime that stops a send.

When a customer withdraws consent, which systems find out? All connected platforms, in the same request as the withdrawal — not a nightly reconciliation, not a webhook behind a paid tier.

A limitation we'll name rather than let you discover. Our current sync resolves conflicts most-recent-write-wins. For consent specifically this is wrong: a later sync could overwrite an earlier withdrawal. We are correcting to last-intent-wins — see §5.

What stops an AI agent from acting without permission? The tool refuses. Agents operate under a signed, spend-capped, revocable, subject-bound mandate; the tool boundary declines to execute without one. The refusal is in the tool, not in a prompt — a prompt is guidance, not a control, and a model can be talked out of guidance. And you can verify any mandate yourself, offline, at `crm-sync.dev/verify` — no account, against our published public key.

Is the enforcement the same across surfaces? Yes. The chat interface and the agent interface call the same `/mcp` tools. Swap the model or the client; the gate holds.

---

3 · ACCESS — THE QUESTION WE'D ASK IF WE WERE YOU

Who on your team can pull a customer's full consent history right now — without a ticket, without an admin, without a license upgrade?

On most stacks the answer is nobody. The record exists, thirty feet away, behind a permissions model, a paid tier, and a person with a full queue. Which means the person accountable for producing the record has no access to it, and the person with access has no accountability for producing it.

A control only one person can exercise is not a control. It's a bottleneck with a compliance label on it.

Our answer: the data subject sees their own record — consent history, which systems hold their data, orders and returns on one timeline. No ticket, no admin, no tier. A DSAR that answers itself is one your DPO never has to file a request to answer.

---

4 · US, AS A VENDOR

"You're small. Why would we depend on you?" Fair, and the honest answer is not "trust us." It's:

- Your data is in your Xano instance and your Cloudflare account. We are not the custodian of your evidence.
- Our attack surface is small enough to enumerate — every route classified and published in the route matrix. **Zero third-party packages in production.** Credentials in one masked store.
- You can audit us. Route matrix, key lifecycle, remediation plan — all public and versioned.

"Isn't Salesforce more secure?" Salesforce's security is genuinely strong — and much of it is a paid tier. Shield is an add-on. Field History Tracking is capped. Event Monitoring costs extra. So the real question is: *which Salesforce did you buy?* A base org without Shield has less audit capability than what ships here by default. That's checkable in your own contract. We'd also gently push on the premise: the configuration surface is easy to use; the enforcement plane is a few hundred lines, published, and small enough to read in full. Ask yourself which you'd rather audit — that, or fifteen years of custom Apex nobody in the building still understands.

What can Cloudflare read? Credentials are stored in Cloudflare KV under Cloudflare-managed encryption at rest. That means: encrypted, isolated per tenant, masked in every API response — and Cloudflare is in your trust boundary. We're not going to blur that. If you need dedicated infrastructure and full data isolation, that's the Private Worker tier.

What PII leaves the system? Raw PII never leaves the Worker. Adobe receives SHA-256 hashes. GA4 receives no PII at all — only tag categories and consent state.

What about card data — the PAN, the CVV? Neither ever reaches us. Payments are tokenized by the wallet (Apple / Google / Samsung Pay) or by Stripe in the browser before anything touches our servers; we settle against a gateway token, never a card number. We checked this by construction, not by policy: no field named for a card number or CVV exists anywhere in the code; **no table in our data store has a column that could hold one** (we read the schema of all 70); nothing logs one; and the reconciliation ledger is id/ref-only by design. Cardholder data is therefore out of scope of this system by construction (an SAQ A shape) — you confirm your own SAQ level with your acquirer.

Bus factor. Handover. What happens if you disappear? A documented key-rotation ceremony, a RACI, and a normative agency-to-client handoff procedure — all public. The enforcement plane is deliberately small so that it can be handed over.

Kill switch? Disable any partner integration with one authenticated config call. No code change, no deploy, under 30 seconds. On most stacks, disconnecting a compromised integration is a release.

5 · KNOWN LIMITATIONS AND REMEDIATION DATES

Nobody who is bluffing publishes this section.

FINDING	SEVERITY	STATUS
Consent ledger is append-only but not hash-chained	High	Sprint 2
Consent writes are best-effort idempotent, not exactly-once (duplicate records possible)	High	Sprint 2
Conflict resolution is most-recent-write-wins — a later sync can overwrite an earlier consent withdrawal; correcting to last-intent-wins (this is the §2 "limitation we'll name" finding)	High	Sprint 2
Consent Mode v2: `ad_storage`, `ad_user_data`, `ad_personalization` all driven by one marketing flag — not separately electable	Medium	Planned
Platform admin key authenticates tenant routes and cannot yet be disabled per tenant	Medium	Sprint 2
Ed25519 signing private key resides in Cloudflare KV, not a dedicated secrets vault	Medium	Sprint 2
No revocation of a signed mandate faster than its expiry (mandates are short-lived + scoped)	Medium	Planned
CSP headers on embedded pages	Low	Open
`X-Content-Type-Options` on JSON responses	Low	Open

Recently closed.

- `/auth/consent-sync` now requires an authenticated session and binds the subject id to the token — unauthenticated writes return 401 (2026-07-14).
- Per-IP rate limiting on `/auth/login`, `/auth/signup`, `/auth/forgot-password`, `/auth/consent-sync` (2026-07-14).
- Twelve routes moved from open to authenticated on 2026-05-17 — `/admin/init-tag-system`, `/admin/xano-schema`, `/admin/xano-reseed`, `/admin/register-webhooks`, `/admin/webflow-ensure-fields`, `/admin/webflow-sync-test`, `/admin/webflow-test`,

`/admin/shopify-customers`, `/admin/shopify-test`, `/sync/customers`, `/sync/webflow`, `/tags/create`.

THE STANDARD WE HOLD OURSELVES TO

We will not answer a question we haven't fixed. If an honest answer would embarrass us, the response is to fix it — not to word it carefully. One carefully-worded answer poisons the other forty, and you'd find it anyway.

Companion: the full Security & Compliance Posture and Security Audit — route matrix and data-pair contracts.
